Performance Test Report

Guideline reference: §10.1 item 9 (performance test results). Requirements tested: SRS NFR-001 (response time) and NFR-009 (scalability). - **Staging impact:** Test 3 made the staging `course-service` unavailable twice and it was restored by a manual restart each time. In Test 4 (after DDP-#73) the overloaded pod restarted by itself and the public API stayed up.

- **Date:** 2026-10-28
- **Environment:** staging, `https://staging.eduralms.com` (AKS, namespace `edura-staging`: one replica per service, shared Neon Postgres).
- **Client:** one laptop in Sri Lanka, one IP address. The servers are in Azure East US, so about 240 ms of every response is network travel.
- **Tool:** `edura-tests/performance/load_test.py` (asyncio and httpx). Read-only GET requests to public endpoints; nothing was created or changed.

Requirements

ID	Requirement	Target
NFR-001	95% of standard user interface interactions processed and rendered in under 2.0 s under normal load	p95 below 2.0 s
NFR-009	Support at least 500 concurrent user sessions with no more than 20% degradation	500 concurrent users

Test 1: response time (NFR-001)

Method: 20 requests per endpoint, 0.8 s apart (about 75 requests per minute, under the gateway's limit), after one warm-up request that is not counted.

Endpoint	Requests	Errors	p50 ms	p95 ms	p99 ms	Under 2 s
`GET /api/courses/`	20	0	247	1,816	2,742	95.0%
`GET /api/courses/11`	20	0	240	325	429	100.0%
`GET /api/analytics/platform-stats`	20	0	246	319	394	100.0%
`GET /api/enrollments/class-packages`	20	0	246	326	343	100.0%

Result: met for these endpoints. Typical responses take about 250 ms. The course list is the weak point: its p95 is 1.8 s, just inside the target, and its slowest request took 2.7 s. It returns the whole catalogue (20 or more courses per call), so it is the first place to look if the 2 s target is missed. With 20 samples per endpoint, a p95 is one request; treat it as an indication, not a guarantee.

Test 2: concurrent load through the gateway

Method: closed-loop workers sending requests continuously for 15 s per step to `GET /api/courses/11` and `GET /api/analytics/platform-stats`.

Concurrent users	Requests	Errors	Throughput req/s	p50 ms	p95 ms	p99 ms
1	50	8 (16.0%, 8 rate-limited)	3.3	235	310	2,040
5	275	241 (87.6%, 241 rate-limited)	18.0	250	328	812
10	560	527 (94.1%, 527 rate-limited)	36.9	247	370	856

Every error was HTTP 429 from the gateway's rate limit (100 requests per minute per IP, burst 20). **No request failed with a server error (5xx) and none timed out.** Requests that were answered kept a p50 of about 250 ms at every step, so the gateway protected the services as designed.

Test 3: load on one service inside the cluster (NFR-009)

Method: a short-lived load pod in the `edura-staging` namespace sent read-only GET requests straight to `course-service` (`GET /11` and `GET /?limit=20`), stepping up the number of concurrent workers. The pod ran with a 500 m CPU limit.

Two attempts were made, and both ended with the service no longer answering:

Attempt	Steps planned	What happened
1	1, 10, 50, 100, 250, 500 workers, 20 s each	The client recorded errors on every request (the error type was not captured, so the figures are not usable). Afterwards `course-service` stopped answering and its readiness check failed; the public API returned 502 until the pod was restarted by hand.
2	1, 3, 5, 10, 20, 40 workers, 15 s each, with automatic stop on errors	At about 10 to 20 workers the public API stopped answering within 8 s, the run was cancelled, and `course-service` again went unready (readiness probe "context deadline exceeded") and returned 502 until restarted. The per-step figures were lost when the pod was removed.

Result: NFR-009 is not met on staging. A single `course-service` pod stops responding at roughly 10 to 20 concurrent clients, long before 500, and does not recover by itself: it stays unready and the gateway returns 502 until the pod is restarted. This is a result about the cheap staging deployment (one replica, one small shared node), which the team chose to keep costs down; it does not say what the application could do on larger infrastructure.

Likely causes, from the configuration (not yet confirmed by profiling): - one uvicorn worker per pod and synchronous database calls (`create_engine` with the default connection pool) to a remote shared Postgres, so slow queries fill the worker's thread pool; - a 500 m CPU limit, and a node shared by all 11 services, RabbitMQ and Redis (the node was already at 87% of its memory requests); - a readiness probe that fails while the service is busy, which removes the only pod from the load balancer.

During the same period the first request after a restart also took several minutes to be accepted (readiness failed with "connection refused" for about 3 minutes after the pod started), which is a separate start-up cost to look at.

Test 4: the same test after the probes and the second replica (NFR-009)

After DDP-#73 was deployed (startup, readiness and liveness probes on every HTTP service, two `course-service` replicas), the in-cluster load test was repeated with the automatic stop rule (stop when more than 10% of a step's requests fail or its p95 passes 5 s). The load pod sent `GET /11` and `GET /?limit=20` straight to `course-service`. All 18 pods were Running before the test.

Concurrent users	Requests	Errors	Throughput req/s	p50 ms	p95 ms	p99 ms
1	791	0	52.7	14	21	34
3	2,211	0	147.2	19	34	52
5	2,325	0	153.7	28	56	76
10	2,205	0	146.4	44	155	208
20	1,297	0	85.3	219	393	586

A second run (load pod with 1 CPU instead of 500 m, so the client is not the bottleneck) continued upward:

Concurrent users	Requests	Errors	Throughput req/s	p50 ms	p95 ms	p99 ms
20	1,652	0	108.3	94	443	652
40	652	70 (10.7%, all read timeouts)	29.1	236	10,003	10,032

The stop rule ended the run at 40 users. What happened next is the main result:

- **The service recovered by itself.** One of the two `course-service` pods stopped answering its health check, was restarted by the liveness probe (restart count 1) and was Ready again about 2 minutes later, with no manual action.
- **The public API stayed up.** Requests through the gateway to `/api/courses/11` returned 200 throughout, apart from one 8 s timeout at the moment the load ended, because the other replica kept serving.

Result for NFR-009 (500 concurrent users): still not met, but much better understood. On the staging hardware, one service handles about 20 concurrent clients with no errors and responses under 0.5 s (p95); at 40 concurrent clients requests begin to time out. That is well below 500. Compared with Tests 2 and 3, a single overload no longer takes the service down until someone restarts it. (The earlier runs failed at roughly 10 to 20 users and needed a manual restart; the change in probes and replicas is the likely reason, but the earlier runs also used a different client, so the two sets are not an exact comparison.)

Response time under load grows with concurrency: the p95 rose from 21 ms with one user to 443 ms with 20, which is far above the 20% limit in the requirement if read as a relative figure, although the absolute response time stays well below the 2 s target of NFR-001.

What these results do and do not show

- **NFR-001 (response time): met** for the endpoints tested, at normal load.

- **NFR-009 (500 concurrent users): not met on staging.** After DDP-#73, one service handles about 20 concurrent clients without errors and starts to time out at 40, and an overloaded pod now restarts by itself (Test 4); before it, a pod failed at about 10 to 20 clients and needed a manual restart. Through the gateway, one IP is limited to 100 requests per minute (Test 2), so a real crowd of students on different networks is limited per IP, not as a whole.
- Only public read endpoints were tested. Sign-in (bcrypt hashing), exam submission and payment webhooks, which are heavier, were not load tested.
- Staging runs one replica per service on a small cluster, so these results are a floor, not a prediction for production.

Gaps and next steps

#	Gap	Suggested action
1	NFR-009 not met (about 20 to 30 concurrent clients per service on staging)	Record it as a known limitation in the final report (the system was deployed on a small budget). Done in DDP-#73: probes and a second `course-service` replica. Not done: a higher CPU limit, several uvicorn workers, more replicas on a larger node, caching or paging of the course list. Re-run Test 4 after any of them.
2	Pod recovery from overload	Done in DDP-#73 (liveness probe), shown in Test 4.
3	Course list p95 is 1.8 s	Add paging or caching to the course list in a later release (application change, deferred).
4	Heavy endpoints not tested	Add sign-in, exam start and submit using a throw-away account.
5	Report is Markdown; deliverable list asks for PDF	Export as `performance-report.pdf` for the final submission.